

Laboratorio 2 Parte 2

Descripción de la práctica:

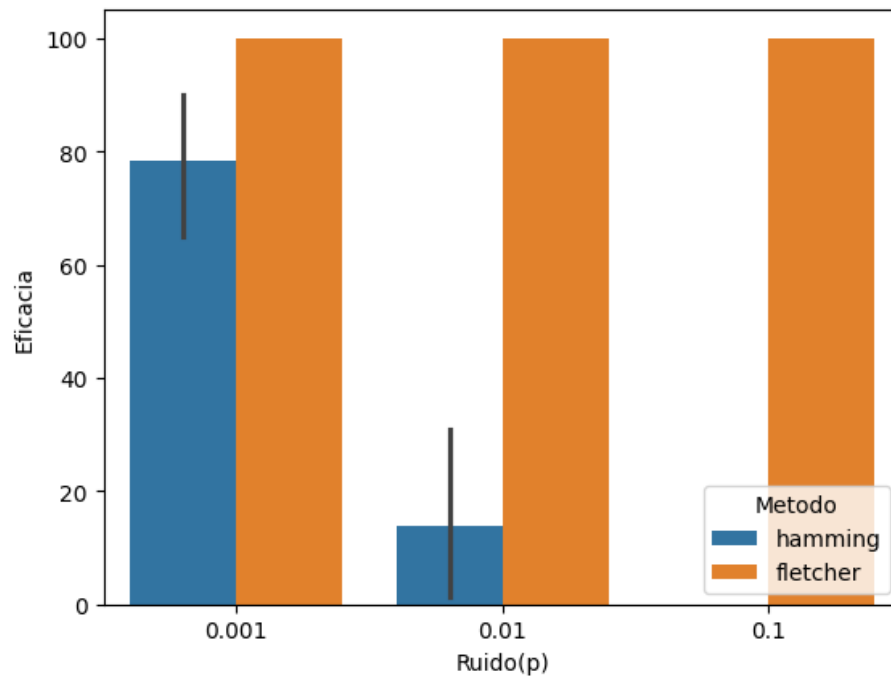
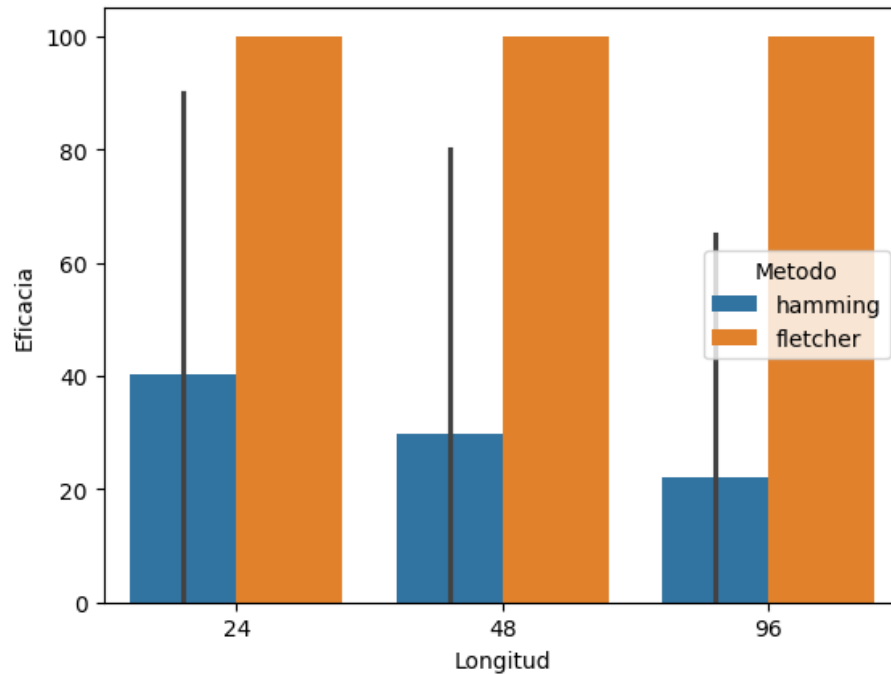
La práctica consiste en utilizar los dos algoritmos desarrollados en la parte 1 y que se pueda comunicar el emisor con el receptor. Se pide un carácter para que este luego se vuelva una cadena de bits, después se tiene un porcentaje de que la cadena de bits tenga un error y luego esta se manda al receptor. Este lee la cadena, decodifica el mensaje y devuelve el carácter que se obtiene de esa cadena de bits.

Las pruebas realizadas consistieron en enviar 1,000 mensajes codificados mediante ambos métodos considerados, Hamming y Fletcher, desde un cliente de Python a un servidor en Julia. Se trabajó con tres tasas de error: 0.1, 0.01, 0.001, con tres longitudes de mensaje: 24, 48, 96. Por simplicidad se trabajó con múltiplos del alfabeto, es decir con 1, 2 y 4 repeticiones de “abcdefghijklmnopqrstuvwxyz”. Se calculó la proporción de los mensajes corregidos correctamente para el caso Hamming y los errores identificados para el caso Fletcher.

Resultados:

```
[(base) ~/Desktop/REDES ➤ python lab2_parte2.py  
Ingresar mensaje: Esta es una prueba  
0: hamming  
1: fletcher  
0  
Enviado a Julia
```

```
[(base) ~/Desktop/REDES ➤ julia lab2_parte2.jl  
Esperando a Python ...  
0000110010101010011110011110011100100100011010001110001000  
000001001111000000110111000101100111101011101010101011  
Verificando con hamming  
Advertencia: se corrigió error en 7.0  
Advertencia: se corrigió error en 14.0  
Mensaje recibido: Esta es una prueba
```



Discusión de resultados:

¿Qué algoritmo tuvo el mejor funcionamiento y porque?

El método de Fletcher tuvo un mejor funcionamiento dado que cumplía con la detección de errores prácticamente todas las veces. Mientras que la corrección de errores mediante

Hamming no se desempeñó de la misma manera para todos los casos. Por lo que se concluye que el método de Fletcher tuvo el mejor funcionamiento a pesar de que no es capaz de corregir los errores que detecta.

¿Qué algoritmo es más flexible para aceptar mayores tasas de errores y porque?

El algoritmo que es más flexible frente a una gran tasa de error es el método de Fletcher. Se puede ver que mantiene un desempeño casi perfecto de manera uniforme respecto a las tasas de error consideradas. Esto puede deberse a que no sacrifica capacidad de detección para realizar corrección.

¿Cuando es mejor utilizar un algoritmo de detección de errores en vez de uno de corrección de errores y porque? ¿Que consideraciones adicionales habría que tener si optamos por solo detectar errores?

Es mejor utilizar un algoritmo de detección de errores cuando se tiene una tasa considerable de errores en la transmisión. Esto es porque es muy complicado y costoso corregir una gran cantidad de errores. Además, nuestros resultados indican que el método de detección es más estable cuando se introduce ruido considerable. Si solo se detecta errores se debe considerar que el mensaje solo es aceptado o rechazado. Lo que indica que se debe idear un método que permita obtener el mensaje correcto.

Comentario grupal:

Previo al laboratorio, no habíamos utilizado sockets pero esta herramienta de programación nos ayudó a comprender como se comportan las conexiones de información entre un emisor y un receptor que no necesariamente están en la misma área.

Conclusiones:

Cuando se implementa con un mensaje mediante un emisor y un receptor, el algoritmo Fletcher Checksum no pierde mucha eficacia aún con la presencia de ruido. Por lo que se recomienda utilizar este dado que es flexible al poder ser aplicado en varias tasas de error lo cual se presenta en situaciones reales al no poder controlar del todo cuando puede suceder un error por estos medios. Y la utilización de ambos algoritmos por varios intentos permite observar cómo se comportan estos en situaciones similares a las que se podrían aplicar de manera práctica.

Repositorio:

https://github.com/ManuelAlejandroMartinezFlores/Redes_Lab2/tree/Parte-2